How the Collision Check Sees the Robot: Colliders, Point Clouds and a Phantom Hand in OmniReset

Tri Wahyu Guntara
Tutorial notes on the UWLab OmniReset code base

1 Why the Success Test Looks for Collisions

Every reset state that the recorder writes to disk will one day be the first frame of a training episode [1]. The policy starts there and physics takes over. If the hand in that frame already overlaps the peg by a few millimetres, the very first physics step pushes the two apart, the peg jumps, and the episode begins with a kick that the policy never asked for. If the fingers overlap the hole block, the kick can throw the arm. A library of such states teaches the policy that the world is jumpy and unfair, which is not what we want it to learn.

Physics itself does not forbid these overlaps. A rigid-body simulator tolerates small interpenetrations and resolves them over a step or two, which is why a hold can look perfectly calm in the viewport while the fingers sit a millimetre inside the peg. The success test therefore does its own measurement at the end of a hold. It asks, for each pair that matters, how deep the closest point of one body sits inside the other, and it compares that depth with the small allowance of Table 1.

Pair	Allowed penetration	Why that number
robot against the insertive object	0.5 mm	Fingers that hold a peg touch it, and the solver leaves them a hair inside. Half a millimetre is contact, not a crash.
robot against the receptive object	0	The hand has no business touching the hole block in a recorded state.
insertive against receptive object	1 mm	A partially inserted peg rests on the hole's wall. One millimetre is a resting contact under the solver's tolerance.

Table 1: The three penetration allowances of the reset-state success test.

The code that performs this measurement is the *collision analyzer*, one instance per row of the table, created by `check_reset_state_success` in `mdp/terminations.py`. The rest of this note explains what it measures, how it measures it, and the mistake that was hiding in the first step.

2 Bodies, Colliders and Prims

Three words come up constantly, and they are easy to mix up.

Body
A rigid body is a piece of the world that moves as one unit. The peg is one body. The robot is many bodies, one per link, joined by joints. A body has a pose, which is a position and an orientation, and physics updates that pose every step.

Collider
A collider is the shape physics uses to decide whether two bodies touch. It is attached to a body and moves with it. A collider is usually simpler than what you see on screen. The peg you see is

a textured box, and its collider is a box of eight corners. The hole block's collider is a mesh of 2 444 vertices, because a hole cannot be approximated by anything simpler. A body can carry several colliders, and the Dex1 base link carries a convex hull for its housing.

Prim
A prim is a node in the USD scene file. Everything in the scene is a prim, arranged in a tree with paths that look like directories, for example `/World/envs/env_0/Robot/right_elbow_link`. A prim becomes a body when the file applies the `RigidBodyAPI` to it, and a prim becomes a collider when the file applies the `CollisionAPI` to it. The two labels are independent, and a collider prim normally sits somewhere below its body prim in the tree.

The last point is the one to hold on to, and Figure 1 draws it for the peg. Physics does not care how deep a collider prim sits below its body in the tree. It walks up from the collider until it meets a prim with the `RigidBodyAPI`, and that body owns the collider. Any code that reads the tree itself has to use the same rule, and we will see in Section 5 what happens when it does not.

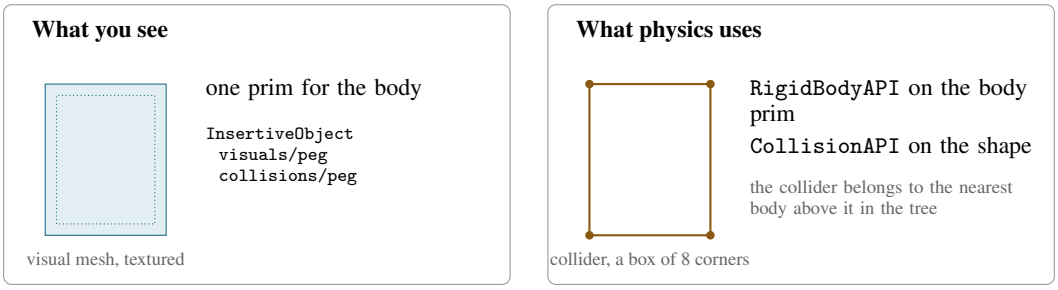


Figure 1: The peg as a prim tree. The visual mesh is what the renderer draws and the collider is what physics touches. Both sit below the body prim, and the collider is owned by that body because it is the nearest prim above it that carries the `RigidBodyAPI`.

3 Measuring a Distance to a Mesh

How far is the hand from the peg? The honest answer involves two complicated shapes, so the analyzer simplifies one side. It scatters sample points over the surface of the robot's colliders and measures each point against the object's mesh. The smallest value over all points is the answer.

Step 1, sample the robot. When the environment starts, the analyzer picks 1 024 points on the surface of each robot link it has been told to watch, spread evenly by farthest-point sampling so that no part of the link is missed. For the G1-D that list is the three shoulder links, the elbow, the three wrist links and the three Dex1 links, ten links and 10 240 points. The points are stored once, in the link's own frame, because the link's shape never changes.

Step 2, move the points to where the link is now. At the end of a hold the analyzer reads each link's current pose and carries the stored points along. A point $\mathbf{q}^{\ell}$ stored in the frame of link ℓ appears in the world at

$$\mathbf{q}^{w} = \mathbf{R}_{\ell}\,\mathbf{q}^{\ell} + \mathbf{t}_{\ell}, \tag{1}$$

where $\mathbf{R}_{\ell}$ and $\mathbf{t}_{\ell}$ are the link's rotation and position. The object has a pose too, so the point is then expressed in the object's frame, $\mathbf{q}^{o} = \mathbf{R}_{o}^{\top}(\mathbf{q}^{w} - \mathbf{t}_{o})$, which lets the object's mesh stay where it was loaded.

Step 3, ask the mesh. Each point is handed to a Warp [2] mesh query, which returns the closest point $\mathbf{c}$ on the mesh surface and a sign s that is $+1$ when the query point lies outside the solid and -1 when it lies inside. The signed distance is

$$d(\mathbf{q}) = s\,\|\mathbf{q} - \mathbf{c}\|, \tag{2}$$

positive for a gap and negative for a penetration. The analyzer keeps the minimum over all points of all watched links, and the pair passes when

$$\min_i d(\mathbf{q}_i) \;\geq\; d_{\min}, \tag{3}$$

with $d_{\min}$ the allowance of Table 1. The query is limited to a radius $d_{\max}$ of 0.5 m. A point with no mesh within that radius returns no closest point, and the analyzer records $+0.5$ for it, meaning "nothing anywhere near". Figure 2 shows the measurement on a finger pad, and the Warp kernel below is Equation (2) line by line, with the scale of the collider undone before the length is taken.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/utils.py lines 293–306

```
            mp = wp.mesh_query_point(mid, q3, max_dist)
            if mp.result:
                if check_dist:
                    closest = wp.mesh_eval_position(mid, mp.face, mp.u, mp.v)
                    local_dist = q3 - closest
                    unscaled_local_dist = wp.vec3(local_dist.x * crs.x, local_dist.y * crs.y,
    local_dist.z * crs.z)
                    delta_root = wp.quat_rotate(coll_rel_quat[index], unscaled_local_dist)
                    dist = wp.length(delta_root)
                    signed_dist = dist * mp.sign
                else:
                    signed_dist = mp.sign
                if signed_dist < best_signed_dist:
                    best_signed_dist = signed_dist
    signs[tid] = best_signed_dist
```

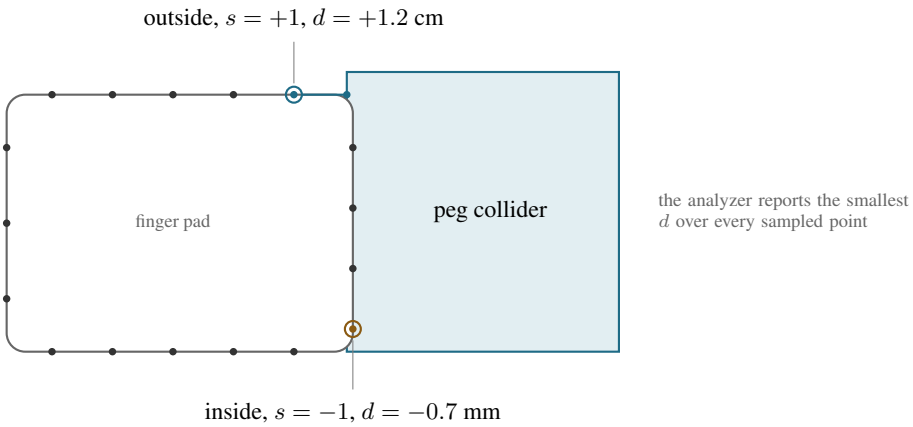


Figure 2: Sample points on a finger pad beside the peg. For each point the mesh query finds the closest surface point and a sign. The pad is pressed a little into the peg, so one of its points lies inside and reports a negative distance. That smallest value is what the pair is judged on, here against the half-millimetre allowance.

The sign-only mode. When an allowance is exactly zero, as for the robot against the hole block, the analyzer skips the length and keeps only the sign (`check_dist` is false in the kernel above). Any point inside makes the pair report -1, and no point inside leaves it at $+0.5$. This is why a debug block shows `min_signed_dist=-1.00000` for that pair instead of a millimetre figure. It is a yes or no, not a depth. The verdict itself is one comparison:

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/collision_analyzer.py lines 190–201

```
        signs = (
            wp.to_torch(sign_w)
            .view(len(self.obstacles), len(env_ids), len(self.body_ids), self.cfg.num_points)
            .amin(dim=0)
        )
        # collision_points = cloud[(signs < 0.0)]
        # for i in range(500):
        #     env.sim.render()
        #     visualizer.visualize(collision_points.view(-1, 3))

        coll_free_mask = signs.amin(dim=(1, 2)) >= self.cfg.min_dist
        return coll_free_mask
```

4 Points Ride on Their Own Link

Step 2 contains an assumption that is easy to overlook. The stored points of a link are moved with that link's pose and nothing else. A point sampled on the fingertip must be stored in the fingertip's frame and moved with the fingertip. If it were stored in the elbow's frame and moved with the elbow, it would be correct only while the wrist stayed exactly as it was when the point was sampled, which is the rest pose of the robot file.

Write the fingertip's pose as the elbow's pose followed by the chain of wrist joints between them, $\mathbf{T}_h = \mathbf{T}_e \, \mathbf{T}_{e \to h}(\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ holds the wrist angles. A fingertip point stored in the elbow frame at the rest pose and carried by the elbow lands at

$$\mathbf{T}_e \, \mathbf{T}_{e \to h}(\boldsymbol{\theta}_0) \, \mathbf{q}^h \quad \text{instead of} \quad \mathbf{T}_e \, \mathbf{T}_{e \to h}(\boldsymbol{\theta}) \, \mathbf{q}^h. \tag{4}$$

The two agree only when $\boldsymbol{\theta} = \boldsymbol{\theta}_0$. Bend the wrist by sixty degrees and the stored copy of the fingertip stays on the straight line through the forearm, about twenty centimetres from the real one on the G1. The analyzer would then measure a fingertip that does not exist.

5 The Phantom Hand

This is exactly what happened on the G1-D and the G1 with the Dex3, and the reason is how their files are laid out. The UR5e file lists its links side by side under the robot prim. The two G1 files were produced by the Kit URDF importer, which nests each link prim inside its parent link, so the tree follows the kinematic chain:

```
Robot
+- torso_link
   +- right_shoulder_pitch_link
      +- right_shoulder_roll_link
         +- right_shoulder_yaw_link
            +- right_elbow_link            <- body, with its own collider below it
               +- right_wrist_roll_link    <- also a body, and also below the elbow prim
                  +- right_wrist_pitch_link
                     +- right_wrist_yaw_link
                        +- right_dex1_base_link
                           +- right_dex1_finger_link_1
                           +- right_dex1_finger_link_2
```

The analyzer builds a link's point cloud by collecting every collider prim below that link's prim. On the UR5e that rule finds exactly the link's own colliders. On the G1 it finds the link's own collider and then every collider of every link further down the chain, because they all sit below the elbow prim in the tree. The elbow's cloud ends up containing the forearm, the wrist and the whole hand, all expressed in the elbow frame at the rest pose and all carried around rigidly by the elbow. The same happens one level down for each wrist link. Figure 3 shows the two rules side by side.

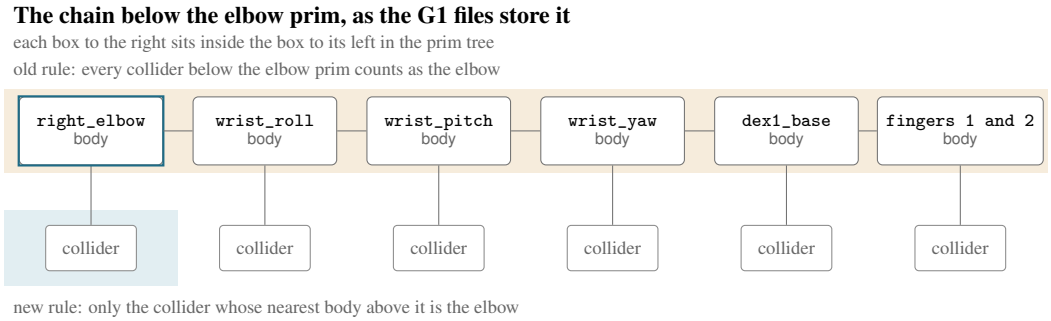


Figure 3: The elbow and everything the G1 files nest below it. The old rule took every collider in the upper band as part of the elbow. The new rule keeps the lower band, the one collider whose nearest body above it is the elbow itself.

How much of each cloud was phantom? Table 2 gives the share of sampled collider area that belongs to descendant links, measured on the two robot files. It is large for every arm link.

Link	G1-D with Dex1	G1 with Dex3
right_shoulder_yaw_link	71%	71%
right_elbow_link	84%	83%
right_wrist_roll_link	88%	88%
right_wrist_pitch_link	83%	82%
right_wrist_yaw_link	84%	83%
gripper base or palm	21%	45%
Dex3 thumb_1	–	100%

Table 2: Share of each link's sampled collider area that belongs to descendant links, before the fix.

Five out of six points on the elbow's cloud were a frozen copy of the wrist and the hand. During a hold the real wrist is bent to put the hand on the peg, so these copies stick out of the elbow and the wrist links along the rest-pose direction, roughly where the hand would be if the arm were straight. They can land inside the hole block, inside the peg or under the table, and the analyzer reports a collision that no one can see in the viewport. Figure 4 draws the situation.

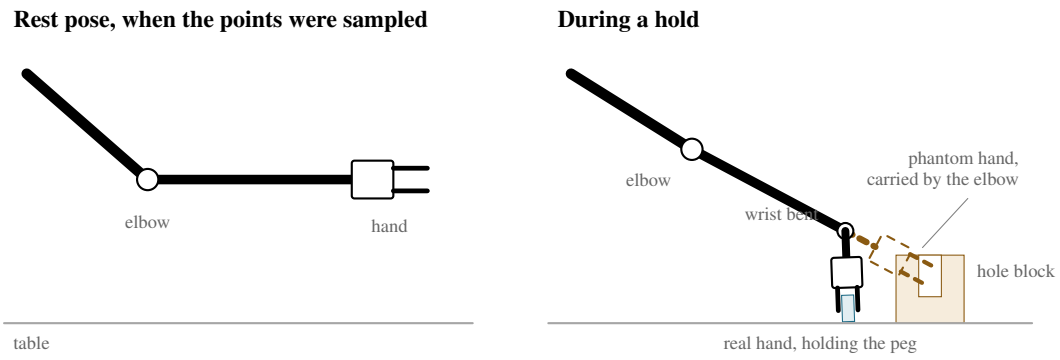


Figure 4: Left, the arm as the robot file stores it, which is when the point clouds are sampled. Right, the arm during a hold with the wrist bent down onto the peg. The real hand is where physics has it. The elbow's point cloud still carries a copy of the hand on the straight line through the forearm, drawn dashed, and that copy can sit inside the hole block or below the table while the real hand is nowhere near.

The debug block that exposed this contained a contradiction. The pair robot against the hole block reported -1, a point inside, while the pair peg against the hole block reported $+0.5$, nothing within half a metre. The peg was in the hand, eleven centimetres from the gripper frame, so the real hand was more than a third of a metre from the hole. Only a point that was not on the real hand could have been inside. Checking the hole mesh itself first ruled out the other suspects, since the mesh is watertight with consistent winding, and Warp's sign gave no false insides on 28 000 test points around it.

6 The Fix, and What It Changes

The rule physics uses is the rule the sampler should use. A collider belongs to the nearest body above it in the tree. The hasher that collects colliders, `RigidObjectHasher` in `mdp/rigid_object_hasher.py`, now applies that rule whenever the prim it was asked about is itself a body. The call site filters the collected colliders before anything is counted or hashed:

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/rigid_object_hasher.py lines 61–69

```
61              coll_prims = get_all_matching_child_prims(
62                  prim_paths[i],
63                  predicate=lambda p: p.GetTypeName() in ("Mesh", "Cube", "Sphere", "Cylinder", "
      Capsule", "Cone")
64                  and p.HasAPI(UsdPhysics.CollisionAPI),
65                  traverse_instance_prims=True,
66              )
67              root_prim = stage.GetPrimAtPath(prim_paths[i])
68              coll_prims = self._own_colliders(root_prim, coll_prims)
69              if len(coll_prims) == 0:
```

and the filter itself climbs from each collider until it meets a body:

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/rigid_object_hasher.py lines 136–162

```
136     @staticmethod
137     def _own_colliders(root_prim: Usd.Prim, coll_prims: list[Usd.Prim]) -> list[Usd.Prim]:
138         """Drop the colliders that belong to another rigid body nested under ‘‘root_prim‘‘.
139
140         A robot converted by the Kit URDF importer nests each link prim inside its parent link, so
      every
141         collider of the forearm, the wrist and the hand also lies under the elbow prim. Sampled as
      part of the
142         elbow they would be frozen at the rest pose and carried around with it. When the root is
      itself a rigid
143         body, only the colliders whose nearest rigid-body ancestor is that root are kept. A root
      without the
144         rigid-body API (an object wrapper prim) keeps every collider under it, as before.
145
146         Args:
147             root_prim: The prim the colliders were collected under.
148             coll_prims: The colliders found under ‘‘root_prim‘‘.
149
150         Returns:
151             The colliders owned by ‘‘root_prim‘‘ itself.
152         """
153         if not root_prim.HasAPI(UsdPhysics.RigidBodyAPI):
154             return coll_prims
155         own = []
156         for prim in coll_prims:
157             parent = prim.GetParent()
158             while parent.IsValid() and parent != root_prim and not parent.HasAPI(UsdPhysics.
      RigidBodyAPI):
159                 parent = parent.GetParent()
160             if parent == root_prim:
161                 own.append(prim)
162         return own
```

For the elbow, the climb from the wrist's collider meets the wrist body first, so that collider is dropped. The climb from the elbow's own collider meets the elbow, so it is kept. Objects are untouched, because their prim path points at a wrapper without the body label, and the UR5e is untouched because its tree was flat all along.

Three things follow from the change.

- The robot analyzers on both G1 robots now measure the real arm. A collision reported by the success test is a collision you can find in the viewport.
- The grasp sampler's own collision filter used the same hasher on the Dex3 and Dex1 hand files, which nest the same way, so it was also judging phantom finger copies. Its yield and the depth of the recorded grasps are worth a second look.
- The success rates measured for the G1 reset-state families before the fix included rejections for phantom collisions. The funnel counts should be taken again.

A tell for the future: a link that reports many more colliders than it visibly has is the signature of this layout. The inspection that produced Table 2 counts, for every body prim, the colliders whose nearest body is a different one. On a flat file every count is zero.

7 Reading a Debug Block

With `debug=True` on the success term, every hold that reaches its final step prints one block. Here is the block that started this investigation, from the in-air grasped family on the G1-D, read line by line.

```
[check_reset_state_success] env 0: success=False
  abnormal_gripper_state=False (any joint > 2x vel limit)
  gripper_orientation_within_range=False approach_z=-0.3401 (< -0.5)
  stability_reached=True counter=30/5
  excessive_pose_deviation=False
    InsertiveObject pos_deviation=0.0082 (<= 0.05)
    ReceptiveObject pos_deviation=0.0000 (<= 0.05)
    Robot pos_deviation=0.0020 (<= 0.1)
  pos_below_threshold=False min_pos_z=0.8702 (>= 0.835)
  collision_free=False
    robot-insertive_object min_signed_dist=-0.00027 (>= -0.0005)
    robot-receptive_object min_signed_dist=-1.00000 (>= 0.0)
    insertive_object-receptive_object min_signed_dist=0.50000 (>= -0.001)
  fingers_blocked=True open_fraction=[0.237, 0.292] mean=0.265 (0.08 < mean < 0.9)
  object_near_ee=True object_ee_dist=0.1075 (< 0.2)
  at_grasp_pose=True pos_err=0.0011 (< 0.015) ang_err=0.0021 (< 0.26)
```

abnormal_gripper_state
: No joint of the robot moved faster than twice its velocity limit at the final step. A violent step would show here.

gripper_orientation_within_range
: The Dex1's approach axis has a world z component of -0.34, which is seventy degrees from pointing straight down. The rule asks for sixty degrees or less, so this clause failed. The hold itself was fine, and this rejection is a policy of the family, not a flaw in the grasp.

stability_reached
: Everything was quiet for all thirty steps, well past the five required.

excessive_pose_deviation and pos_below_threshold
: The peg moved 8 mm since the reset, the hand 2 mm, the hole block not at all, and nothing dropped below two centimetres under the table top. All well inside their limits.

collision_free
: The hand sits 0.27 mm inside the peg, which is ordinary contact and under the half-millimetre allowance. The hole block reports a point inside in the sign-only mode, and the peg reports nothing within half a metre of the block. That contradiction is Section 5.

fingers_blocked, object_near_ee, at_grasp_pose
: The fingers stopped at a quarter of their travel, so something is between them. The peg is 11 cm from the gripper frame. The hand is 1 mm and 0.1 degrees from the dataset's grasp pose. These three together say the peg is held exactly as sampled.

So this block describes a real grasp that failed for one legitimate reason, its approach angle, and one phantom reason, a collider copy inside the hole block. After the fix only the first remains, and

whether a seventy-degree approach should count as a valid in-air state is a question about the family, not about the analyzer.

8 Check Your Understanding

1. The peg analyzer reports $+0.5$ and the robot analyzer reports -1. Why can both not be true at once for a hand that holds the peg?

   Answer. $+0.5$ means no point of the peg has any hole-block mesh within half a metre. The hand is within about eleven centimetres of the peg, so no point on the real hand can be inside the block. A -1 must then come from a point that is not on the real hand.

2. Why does the fix not change anything for the peg or the hole block?

   Answer. Their prim paths point at a wrapper prim without the `RigidBodyAPI`. The new rule only applies when the queried prim is itself a body, so for objects every collider below the wrapper is kept, as before.

3. Why did the UR5e never show the problem?

   Answer. Its file lists the links side by side under the robot prim. No link prim contains another link, so collecting every collider below a link already gives only that link's own colliders.

4. A hold shows `robot-insertive_object min_signed_dist=-0.0009`. Is that a crash?

   Answer. It is a 0.9 mm penetration, which fails the 0.5 mm allowance, but it is the kind of depth a soft fingertip pressed on a peg reaches when the solver settles. It says the grasp is squeezing hard rather than that the hand is passing through the peg. Whether to loosen the allowance is a judgement about how the fingers are driven.

Files involved. `mdp/collision_analyzer.py`, `mdp/rigid_object_hasher.py`, `mdp/utils.py` (`sample_object_point_cloud`, `get_signed_distance`) and `mdp/terminations.py`. The measurements come from the robot USD files under `source/uwlab_assets/data/Robots/Unitree`. Written 30 October 2026 for the `feature/forge-omnireset` branch.

References

[1] UW Lab. Emergent dexterity via diverse resets and large-scale reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.15789.

[2] M. Macklin. Warp: A high-performance python framework for gpu simulation and graphics. NVIDIA GPU Technology Conference (GTC), 2022. https://github.com/NVIDIA/warp.